# A. Artifact Appendix

## A.1 Abstract

The testing methodology described in Sections 3 and 4 categorizes websites based on (i) disclosure of tracking techniques in cookie-banner text and (ii) the prevalence of sophisticated tracking mechanisms observed in practice, such as browser fingerprinting. The artifact uses zero-shot prompting with LLMs to analyze banner text, identify the tracking techniques disclosed, and determine the corresponding level of disclosure. It further provides the BannerClick-based crawling infrastructure for large-scale measurement of tracking mechanisms before and after interaction with cookie banners. The disclosed techniques are then compared with the tracking mechanisms observed during crawling to assess whether the banner disclosures accurately reflect the tracking practices implemented by each website.

## A.2 Artifact check-list (meta-information)

- **Algorithm:** Zero-shot prompting with contextual understanding
- **Program:** BannerClick, modified OpenWPM, Python analysis scripts, Jupyter notebooks
- **Model:** GPT-4.1 (`gpt-4.1-2025-04-14`)
- **Data set:** Tranco Top 100K domains
- **Run-time environment:** Conda environment, Python, Linux OS
- **Hardware:** 8GB RAM , 4 cores, x86-64(AMD64) Architecture
- **Execution:** BannerClick crawling; Jupyter notebooks for preprocessing and graph generation (`Preprocessing.ipynb`, `Graphs.ipynb`)
- **Output:** Crawl data, processed CSV files, tracking measurements, statistical analyses, figures
- **Experiments:** Pre/post-banner tracking, tracking disclosure
- **How much disk space required (approximately)?:** 1.4–2.2 GB per 100 websites
- **How much time is needed to prepare workflow (approximately)?:** 20–30 minutes
- **How much time is needed to complete experiments (approximately)?:** $\sim$3 hours per 100 websites
- **Publicly available:** Yes
- **Code licenses:** GPLv3
- **Data licenses:** CC BY 4.0
- **Workflow automation framework used?:** BannerClick, modified OpenWPM
- **Archived (provide DOI)?:** `https://doi.org/10.5281/zenodo.21708043`

## A.3 Description

Large-scale measurements were conducted on multiple AWS instances located in Germany with the Tranco Top 100K websites processed in small batches. BannerClick, implemented on a modified OpenWPM framework, crawls each website, identifies cookie banners, and records tracking observations before and after banner interaction. The resulting crawl data, including banner text and tracking observations, are stored in a csv files and `crawl-data.sqlite` database respectively. The GitHub repository contains the complete source code for BannerClick, the modified OpenWPM framework, and the associated analysis pipeline. Additional details on the crawling procedures are provided in the `README.md` files of the code and data repositories.

### A.3.1 How to access

The datasets are publicly available in the data repository, which also provides references to the analysis code: `https://doi.org/10.5281/zenodo.21708043`

The source code can be accessed by cloning the GitHub repository:

```
git clone https://github.com/is-shah/Beyond-Cookies.git
```

Detailed instructions for installation, execution are provided in the repository's `README.md` file.

### A.3.2 Software dependencies

The code has been produced and run on a Linux os. The installation procedure is written for conda environment and the relevant packages must be installed using `./install.sh` file in source code .

### A.3.3 Data sets

Our measurements are conducted across the Tranco Top 100K domains. The complete crawl datasets generated by our tool, stored as `crawl-data.sqlite` files, are publicly available at `https://doi.org/10.5281/zenodo.21708043`. These raw crawl datasets serve as the primary input for the subsequent preprocessing and analysis pipeline used to derive the paper's main findings.

### A.3.4 Models

We use GPT-4.1 (`gpt-4.1-2025-04-14`) with zero-shot prompting to categorize tracking disclosures in cookie-banner text. To reproduce the categorization, first provide an OpenAI API key:

```
export OPENAI_API_KEY=<your-key>
```

The banner text to be classified should be provided through the `banner_text.csv` input specified in `llm.py`. If the banner text is not in English, run `translate.py` first to translate the text into English. The categorization can then be executed using:

```
python llm.py
```

The resulting classifications are used in the subsequent analysis pipeline. For reference, `total.json` in `subcode` contains the results from the original model runs used in the paper.

## A.4 Installation

The complete installation and setup procedure is provided in the project `README.md`: `https://github.com/is-shah/Beyond-Cookies/blob/main/README.md`. The instructions cover all required dependencies, including creation and configuration of the OpenWPM Conda environment, installation of the necessary packages, and Firefox browser setup required to execute the BannerClick crawling workflow.

## A.5 Experiment workflow

The experiment workflow consists of two parallel stages: banner-text categorization and tracking-measurement analysis.

1. Set up the Conda environment and the corresponding Firefox browser configuration as described in the repository's installation instructions.
2. Start the BannerClick crawler using `run.py` with the required command-line arguments. Once crawling is completed for the target dataset, the resulting crawl data are available in the corresponding crawl directory.
3. For banner-text categorization, extract the `banner_text.csv` file from the crawl output. If the banner text is not already in English, first process it using `translate.py`. The resulting

English banner text is then provided to `llm.py` for zero-shot categorization of tracking disclosures.

4. For tracking-measurement analysis, locate the corresponding `crawl-data.sqlite` file in the appropriate `datadir/test-artifact/` subdirectory. Open `Preprocessing.ipynb`, specify the database path using the `DB_PATH` variable, and execute the notebook.
5. The preprocessing workflow generates two analysis-ready CSV files, `js.csv` and `js_cookies.csv`, containing the processed JavaScript and cookie-related tracking observations.
6. Use the output CSV files as inputs to `Graphs.ipynb`. Attach the required input files to the notebook and execute the notebook to generate the analyses, statistics, and figures used in the paper.

### A.6 Evaluation and expected results

The results can be found in the data repository . The results can be verified by running a Jupyter Notebook server and executing the Notebook 'Graphs.ipynb'.